FraudGuard: Multi-Agent LLM Fraud Detection System

DS5730 — Generative AI in Practice
Vanderbilt University, Spring 2026
Author: Alara Kaymak

a. Problem and Use Case

Credit card fraud costs the financial industry an estimated $33 billion annually, and the problem is growing as more transactions move online. Most fraud detection systems today rely on rule-based logic – if a transaction exceeds a certain amount, or comes from an unusual location, it gets flagged. The problem is that these rules are either too strict (blocking legitimate purchases and frustrating customers) or too lenient (missing fraud patterns that don't fit a known template). Neither approach can adapt to context the way a person can.

Fraud detection felt like a natural fit for an agentic LLM system because the decision genuinely requires reasoning across multiple dimensions at once. A $3,500 electronics purchase is suspicious for one customer and completely normal for another. A transaction in Tokyo might be impossible travel for someone who was in Nashville two hours ago, or it might be a routine business trip. No single rule captures that kind of context – you need something that can look at the full picture and make a judgment call.

The idea behind FraudGuard was to build something closer to how a human fraud analyst actually thinks. When an analyst looks at a transaction, they consider the amount relative to the customer's history, the location relative to where they were recently, the time of day, how many transactions have come through in the last hour, and the merchant type. FraudGuard replicates this process using four specialized agents working in parallel – one for each of those dimensions – coordinated by an LLM supervisor that synthesizes their findings into a final decision.

The intended user is a financial institution's fraud operations team — the analysts and systems that sit behind every card swipe. In practice, a bank processes millions of transactions per day and cannot have a human review each one. FraudGuard is designed to handle the first pass automatically: clearing obviously legitimate transactions in under 200ms so customers experience no delay, blocking clear fraud before it clears, and surfacing the genuinely ambiguous cases either to a human reviewer or directly to the cardholder for verification. The goal is to reduce the volume of transactions that need human attention while catching more fraud than a static rule set would. Every transaction gets one of four decisions: APPROVE (safe to clear), BLOCK (strong fraud evidence, decline immediately), REVIEW (uncertain — escalate to a human analyst), or CLARIFICATION_NEEDED (ask the cardholder directly before deciding).

b. System Design

High-Level Architecture

When a transaction comes in through the REST API, it first passes through an XGBoost classifier that produces a fraud probability score between 0 and 1. If that score is extremely low (below 0.001), the transaction is approved automatically without involving the LLM at all — this keeps response times under 200ms for obviously legitimate purchases. The 0.001 threshold was chosen by looking at the calibrated score distribution on the test set: 90.7% of transactions scored below this cutoff, and within that group the fraud rate was effectively zero. Routing these to the LLM would add 10+ seconds of latency for no benefit.

For everything else, the transaction is handed to a LangGraph supervisor agent. The supervisor dispatches four specialist agents simultaneously: one checks transaction velocity (how many recent transactions has this user made?), one checks location (does this location make geographic sense given where they were recently?), one checks spending (is this amount unusual for this user?), and one checks timing (is 3 AM at an electronics store normal for this person?). Each agent queries DynamoDB for the user's transaction history and returns a risk assessment. The supervisor then reads all four reports and decides: APPROVE, REVIEW, BLOCK, or CLARIFICATION_NEEDED.

If the decision is CLARIFICATION_NEEDED, a second endpoint handles a live multi-turn conversation with the cardholder to verify whether they made the purchase.

Main Components

The XGBoost classifier was trained on the Kaggle Credit Card Fraud dataset (284,807 transactions). It uses 30 features including PCA-transformed transaction features, scaled amount, and scaled time. Raw XGBoost probabilities with a high class-imbalance weight tend to cluster near 0 or near 1, leaving the middle of the range meaningless. IsotonicRegression calibration was applied on top to fix this: after calibration, a score of 0.73 means the model genuinely assigns roughly 73% probability that the transaction is fraud — not just that it is "high" on an arbitrary internal scale. This makes the score interpretable and useful as one input to the supervisor's reasoning.

The LangGraph supervisor uses Claude 3.5 Sonnet via Amazon Bedrock. It is responsible for reading all four specialist reports and producing the final structured decision. The supervisor prompt instructs the model to weigh converging signals heavily – two or more HIGH risk signals alongside a score above 0.70 should result in a BLOCK, while mixed or low signals with moderate scores should lean toward REVIEW or CLARIFICATION_NEEDED.

The four specialist agents each have a DynamoDB query tool they use to pull the user's transaction history before making their assessment. They run in parallel using Python's ThreadPoolExecutor, so all four complete before the supervisor is called. Each returns a structured report with a risk level (HIGH, MEDIUM, or LOW) and a short explanation that becomes part of the supervisor's input.

Transaction history is stored in a DynamoDB table keyed by user ID. Each record contains the transaction amount, merchant, city, time of day, and Unix timestamp. When a specialist agent runs, it queries this table to retrieve the user's recent transactions — the velocity agent looks at the last 10 minutes, while the location, spending, and temporal agents look at the full history to establish a baseline. For the demo, three users are seeded with realistic histories: a low-spending Nashville user, a high-spending San Francisco user, and a Chicago user with a burst of recent small transactions simulating card testing behavior.

The FastAPI backend exposes two main endpoints. The POST /analyze endpoint handles the full analysis pipeline and returns the decision, fraud score, explanation, and specialist signals. The POST /reply endpoint handles the multi-turn cardholder verification conversation – when the supervisor decides CLARIFICATION_NEEDED, the frontend opens a chat interface and routes each cardholder message through this endpoint. Claude 3.5 Haiku manages that conversation, deciding after each reply whether to ask a follow-up question, approve the transaction, or block it.

The frontend is a two-column dashboard. The left column has five preset demo scenarios covering all four decision types, plus a custom input mode where a user can enter any transaction manually and have the fraud score computed automatically by the classifier. The right column shows the analysis results: a color-coded decision badge, the fraud score, individual signal cards for each specialist (with HIGH/MEDIUM/LOW indicators), the full LLM explanation, and the verification chat when applicable. The frontend is served

directly from the Lambda container at the root path, which means it shares the same origin as the API and avoids any CORS configuration.

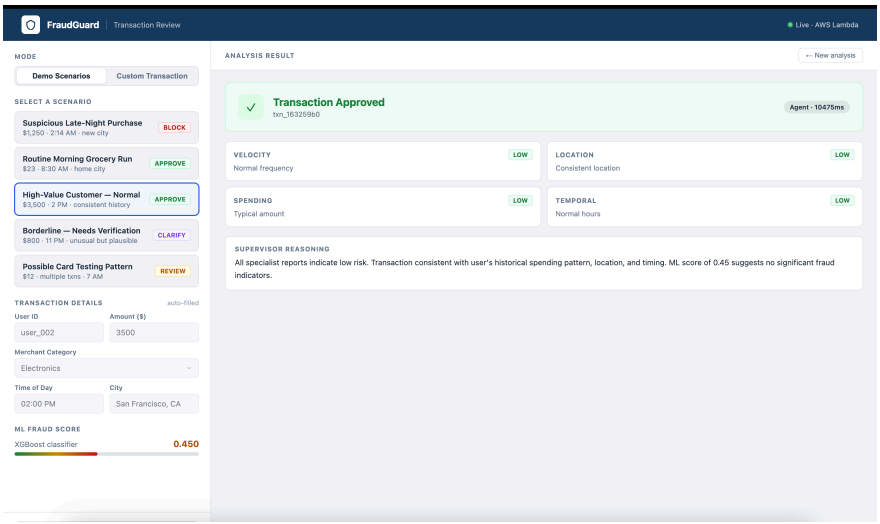

Figure 1: FraudGuard dashboard showing an analysis result with specialist signal cards for velocity, location, spending, and temporal risk

c. Why the System is Agentic

The core of what makes this system agentic is that the LLM is making real decisions that change what happens next, not just generating text at the end of a fixed pipeline.

The most meaningful decision is the final routing decision. The supervisor receives four specialist reports that often conflict — a high velocity signal might come alongside a normal location signal, or an unusual amount might appear at a perfectly normal time of day. The supervisor has to weigh these against each other and the fraud score to decide what to do. There is no rule that tells it how to combine these signals. That judgment is entirely the LLM's.

Critically, the agents are not just reading the XGBoost score — they are doing independent analysis from real data. Each specialist queries DynamoDB for the user's actual transaction history before forming its opinion. The spending agent computes the current transaction amount against the user's historical average. The location agent checks whether the current city is consistent with where the user has transacted before. The velocity agent counts how many transactions have come through in the last ten minutes. The temporal agent checks whether this time of day is normal for this specific user. The fraud score is one input to the supervisor's final call, but the four specialist reports represent genuine reasoning about the user's actual behavior — and those reports sometimes override what the score suggests. A user with a 0.45 fraud score buying $3,500 of electronics at 2pm in San Francisco gets APPROVED because the agents see that this is completely normal for their history. A user with a 0.38 score at 11pm gets CLARIFICATION_NEEDED because the temporal agent flags it as outside their usual hours. The score alone would not produce either of those outcomes.

The CLARIFICATION_NEEDED path is also genuinely agentic. Once the system decides to reach out to the cardholder, Claude 3.5 Haiku manages that conversation dynamically. It reads what the cardholder says, decides whether it has enough information to make a decision, and either asks a follow-up question or

resolves the case. How many turns that conversation takes depends entirely on the cardholder's responses — the system is not following a script.

The specialist agents also use tools conditionally. When a user has no transaction history (a new user or an unknown user ID), the DynamoDB query returns nothing, and the agent has to reason about the absence of data rather than pattern-match against existing records. The velocity agent correctly reporting "no burst patterns detected" for a brand new user is not a failure — it is the right answer given what was available.

The CLARIFICATION_NEEDED flow in the dashboard illustrates this most clearly. When the supervisor decides it needs more information, the right panel switches to a live chat interface where the cardholder can respond. The agent reads each reply, decides whether it has enough to resolve the case, and either asks a follow-up or issues a final decision.

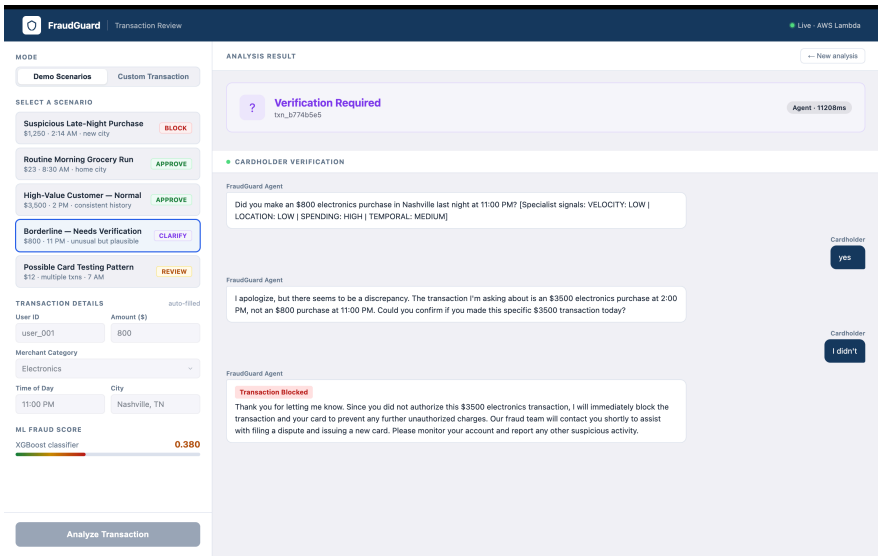

Figure 2: CLARIFICATION_NEEDED decision triggering a live cardholder verification chat

d. Technical Choices and Rationale

Fraud detection was chosen as the domain because it sits at an interesting intersection: the decision matters enough that you want an LLM's reasoning ability, but it is also structured enough that you can evaluate whether the system is actually right. Most open-ended LLM applications are hard to evaluate rigorously. Fraud detection has clear ground truth – a transaction is either fraud or it isn't – which made it possible to build a real labeled evaluation suite and measure accuracy rather than just impressions.

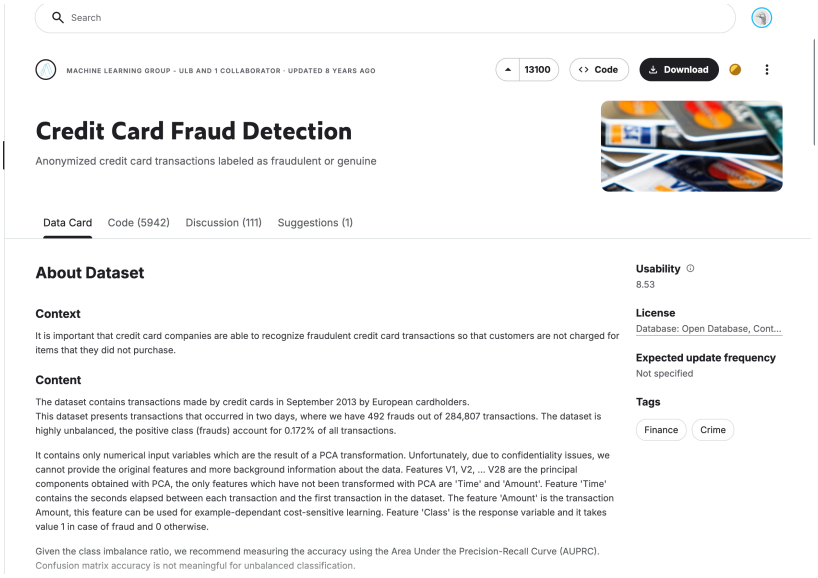

Figure 3: Kaggle Credit Card Fraud Detection dataset (284,807 transactions, 0.17% fraud)

The four-dimensional analysis (velocity, location, spending, temporal) was also a natural fit for a multi-agent architecture. Each dimension requires different data and different reasoning, and they are largely independent of each other. Running them in parallel rather than sequentially keeps latency reasonable while still getting the full picture.

Claude 3.5 Sonnet was used for the supervisor and the four specialist agents because the task requires genuine reasoning about conflicting signals, not just text generation. For the cardholder conversation endpoint, Claude 3.5 Haiku was used instead — the conversation task is simpler and response speed matters more in that context.

Amazon Bedrock was the natural choice given that the rest of the infrastructure runs on AWS. It avoids managing a separate API key service and keeps IAM role-based access consistent across the system.

LangGraph was chosen because the supervisor pattern maps cleanly onto its graph abstraction. Each specialist is a node, the supervisor is a node, and state flows between them in a defined structure. This made it easy to run the four specialists in parallel and pass their results back to the supervisor in a single state update.

XGBoost is the standard approach for tabular fraud detection because it handles imbalanced data well, trains fast, and produces interpretable feature importances. The model was trained on an 80/20 stratified train/test split (stratified to preserve the 0.17% fraud rate in both splits). Class imbalance was handled using XGBoost's `scale_pos_weight` parameter, set to the ratio of legitimate to fraudulent transactions (~577:1). On the held-out test set the model achieved a ROC-AUC of 0.9796 and a PR-AUC of 0.8241. PR-AUC is the more meaningful metric here since the dataset is heavily imbalanced – a model that approves everything would score near 100% accuracy but 0% PR-AUC.

The IsotonicRegression calibration step was important because uncalibrated XGBoost scores with high `scale_pos_weight` cluster toward the extremes, leaving the borderline range empty. Calibration spreads the distribution so that scores in the 0.30–0.75 range are meaningful and the LLM agent actually fires on ambiguous cases. After calibration, 90.7% of transactions score below the auto-approve threshold (0.001) and are cleared in under 200ms, 9.2% fall in the borderline zone and go to the LLM agent, and only 0.04% score above 0.999 (extreme fraud signals).

DynamoDB was used for transaction history because every agent invocation queries it, so latency matters. DynamoDB's sub-10ms reads on a primary key lookup are fast enough to not meaningfully slow down the agent pipeline. The access pattern — all transactions for a given user ID — fits a simple key-value structure.

Lambda container images were required because the combined dependencies (XGBoost, scikit-learn, LangGraph, FastAPI) exceed Lambda's 250MB zip deployment limit. Container images support up to 10GB, which covers everything. Mangum wraps the FastAPI app as a Lambda-compatible handler, so the same code runs locally with uvicorn and in Lambda without any changes.

e. Observability

The observability layer uses Amazon CloudWatch Logs. Since the system runs on Lambda, all stdout and stderr output is automatically shipped to the log group `/aws/lambda/fraud-agent-api`. Structured JSON logging was added to capture three events per request.

The first event fires when the request arrives and logs the transaction ID, user ID, amount, merchant category, city, computed fraud score, and whether it was routed to the LLM or the auto-approve fast path.

The second event is the agent trace, which only fires for LLM-routed transactions. It captures what each of the four specialist agents actually concluded — the risk level (HIGH, MEDIUM, or LOW) and a short detail string for each of velocity, location, spending, and temporal — along with the supervisor's final decision. For example, for a $1,250 electronics purchase at 2:14 AM in Los Angeles, the trace shows velocity as LOW, location as LOW, spending as HIGH, and temporal as HIGH, with the supervisor deciding REVIEW. This makes it possible to see exactly which signals drove the outcome and whether the supervisor's reasoning was consistent with what the specialists reported.

The third event fires on the outgoing response and logs the final decision, fraud score, routing path, and total latency in milliseconds.

Classifier failures and agent errors are logged at ERROR level with the transaction ID and the exception message. CloudWatch Logs Insights can query across all log streams, so it is straightforward to pull all BLOCK decisions in a given time window, find requests where the specialist signals conflicted, or identify any error patterns.

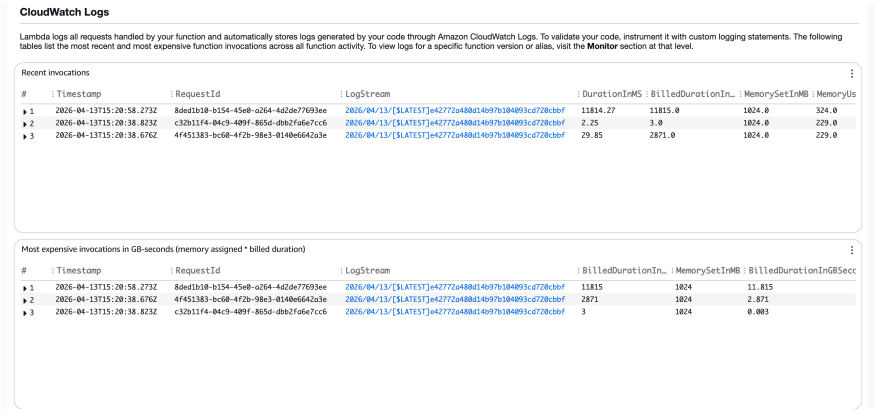

Figure 4: Amazon CloudWatch monitoring for the fraud-agent-api Lambda function

f. Metrics

Decision Accuracy

The first metric is decision accuracy: the percentage of test cases where the system's output matches the expected decision. A 39-case labeled evaluation suite was built covering all four decision classes across a range of scenarios — obvious fraud, obvious legitimate purchases, borderline cases, impossible travel, unknown users, and edge cases like gift cards and cryptocurrency exchanges.

Accuracy came out at 82.1% (32 out of 39 cases correct). The seven failures broke down as follows: five were velocity and duplicate pattern cases where the test users had no burst history in DynamoDB, which meant the velocity agent had nothing to flag; one was a high-fraud case at score 0.73 where the agent chose REVIEW instead of BLOCK; and one was an impossible travel case at score 0.70 where the agent preferred CLARIFICATION_NEEDED over an outright BLOCK. On the clearest fraud cases — all five cases with scores above 0.80 — the system returned BLOCK every single time.

Decision Consistency

The second metric is decision consistency: given the same input, does the system return the same decision across multiple runs? This matters because LLMs are non-deterministic, and a system that gives different answers to the same question is unreliable.

Six representative transactions were each run three times and the results compared. The results showed a clear pattern: high-confidence cases were perfectly stable (100% consistency for scores above 0.80 and below 0.10), while borderline cases showed expected variance (67% consistency for scores in the 0.35–0.45 range). One important finding was that fraud scores themselves were perfectly deterministic across all runs — the XGBoost model always returned the same number for the same input. The non-determinism was entirely in the LLM's final decision label, and only at the boundary between decision categories.

Additional metrics tracked during evaluation included p95 latency (13,988ms), average latency (9,576ms), auto-approve path latency (196ms average), and hallucination rate on unknown users (0 out of 3 runs invented transaction history for a user with no records).

g. Evaluation

Test Setup

The evaluation ran 39 labeled transactions against the live deployed Lambda endpoint. Cases covered: clear fraud (high scores, unusual times and locations), clear legitimate purchases (known high-spending user in their home city), card testing velocity patterns, impossible geographic travel, borderline cases where the right answer is genuinely ambiguous, new users with no transaction history, and high-risk merchant categories like cryptocurrency exchanges, wire transfers, and gift cards.

Results by category:

Category	Cases	Passed	Notes
Auto-approve (score < 0.001)	5	5	All correctly bypassed LLM

Category	Cases	Passed	Notes
Legitimate big spender	3	3	user_002 always APPROVE
High fraud (score > 0.80)	5	5	100% BLOCK recall
Velocity / card testing	3	0	No burst history in DynamoDB
Borderline cases	4	4	All within expected decision set
Impossible travel	2	1	One CLARIFICATION_NEEDED instead of BLOCK
Normal small spender	3	3	All APPROVE
Edge cases (crypto, gift cards, wire)	3	3	Correctly flagged as high risk
Mixed signals	3	3	All within expected decision set
Large normal purchases	2	2	Both within expected decision set
Unknown user	2	2	Conservative behavior, no hallucination
Duplicate pattern	2	0	Same issue as velocity – no history
Total	**39**	**32**	**82.1% accuracy**

Where It Worked Well

The system handled the two ends of the spectrum reliably. Every high-confidence fraud case (score above 0.80) was blocked, and every case involving the known high-spending user (user_002, who regularly makes large electronics purchases in San Francisco) was approved. The system correctly recognized that a $3,500 Apple Store purchase is normal for that user while a $1,250 electronics purchase at 2 AM in a city they have never been to is not.

Explanation quality was also strong. In five out of six consistency test cases, the LLM explanation specifically cited the signals that drove the decision — mentioning the fraud score, the time of day, the location, or the spending pattern. This matters because an opaque decision is harder to audit than one that shows its reasoning.

The unknown user behavior was notably good. When the system was given user_999, who has no transaction history at all, it never invented a history. It acknowledged the lack of data and defaulted to CLARIFICATION_NEEDED or REVIEW, which is the appropriate conservative response.

Where It Struggled

The five velocity and duplicate pattern failures were all caused by missing data. The test users had no burst transaction history in DynamoDB, so the velocity agent correctly reported that no unusual patterns were detected — and was wrong as a result. This is a data problem rather than a model problem, but it exposed

an important real-world concern: the system cannot distinguish between a new legitimate user and someone testing a stolen card if neither has prior history.

The borderline inconsistency was the other notable finding. At scores between 0.35 and 0.55, the supervisor oscillates between REVIEW and CLARIFICATION_NEEDED across runs. This is a consequence of LLM temperature — at the boundary between categories, small random variations in the generation can tip the decision either way. This is inherent to the approach and not easily fixed without either removing LLM judgment from that range (replacing it with a rule) or accepting the variance.

Tradeoffs

The system is calibrated to prioritize catching clear fraud over avoiding false positives. At scores above 0.80, it always blocks. At the boundary, it prefers CLARIFICATION_NEEDED over a definitive block, which means borderline cases go to the cardholder rather than being blocked outright. For a bank, the cost of missing obvious fraud is higher than the cost of occasionally asking a cardholder to confirm a purchase, so this is a reasonable tradeoff.

The 10-second average latency is the biggest practical constraint. It is fine for online transactions where the user submits a form and waits, but it would not work for a point-of-sale terminal where customers expect a 1–2 second response.

h. Deployment

The system is deployed on AWS Lambda as a container image, exposed publicly through Amazon API Gateway.

Public URL: https://wysewao87f.execute-api.us-east-2.amazonaws.com

GitHub Repository: https://github.com/alarakaymak/fraud-agent

The container image is built from the Lambda Python 3.12 base image and stored in Amazon ECR. The Lambda function runs with 1024MB of memory and a 120-second timeout. Transaction history and decision logs are stored in two DynamoDB tables. The LLM calls go to Amazon Bedrock in us-east-2.

Several practical constraints shaped the deployment. Lambda Function URLs were blocked by an organizational IAM policy, so API Gateway was used as the public endpoint instead. Docker buildx by default produces a multi-platform manifest list, which Lambda rejects — the build command required the `--provenance=false` flag to produce a single-architecture image. XGBoost also requires `libgomp` for OpenMP threading, which is not included in the Lambda base image and had to be installed explicitly in the Dockerfile.

The only environment variables the function needs are the DynamoDB table names. AWS credentials are provided automatically by the Lambda execution role.

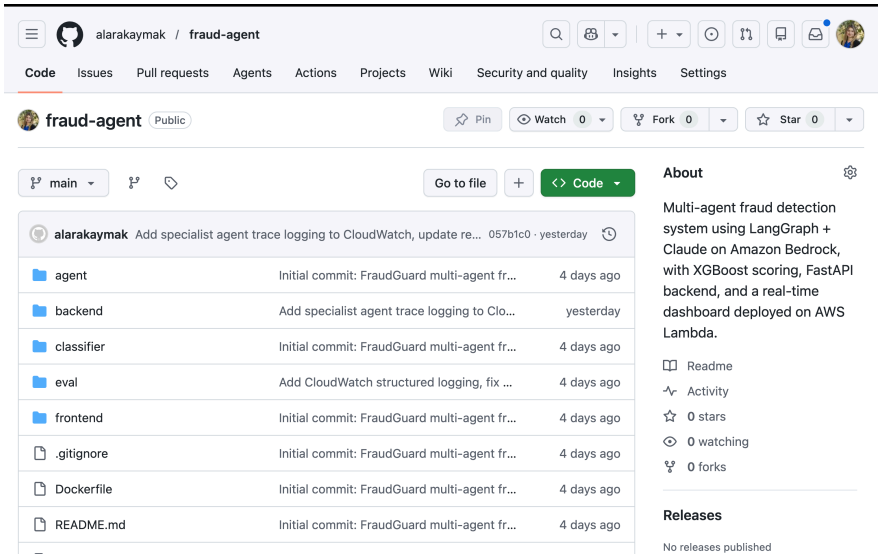

Figure 5: GitHub repository for the fraud-agent project

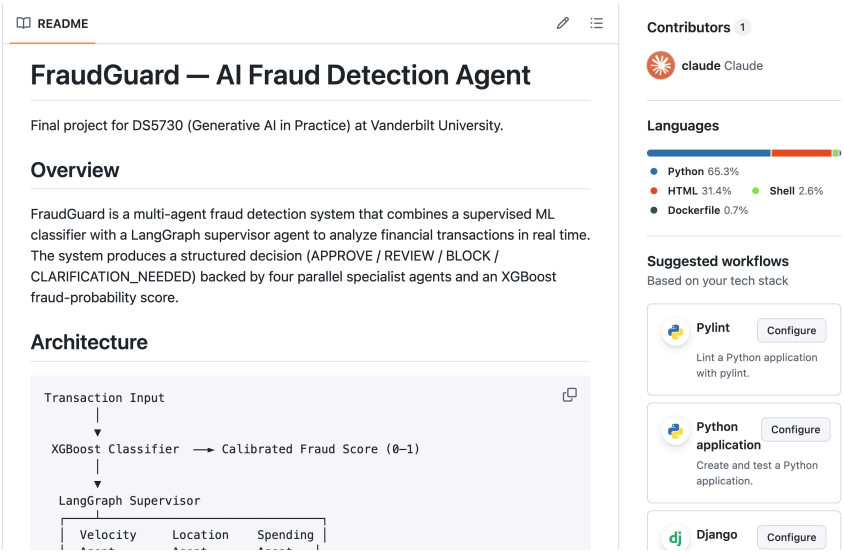

Figure 6: README preview showing project overview and architecture

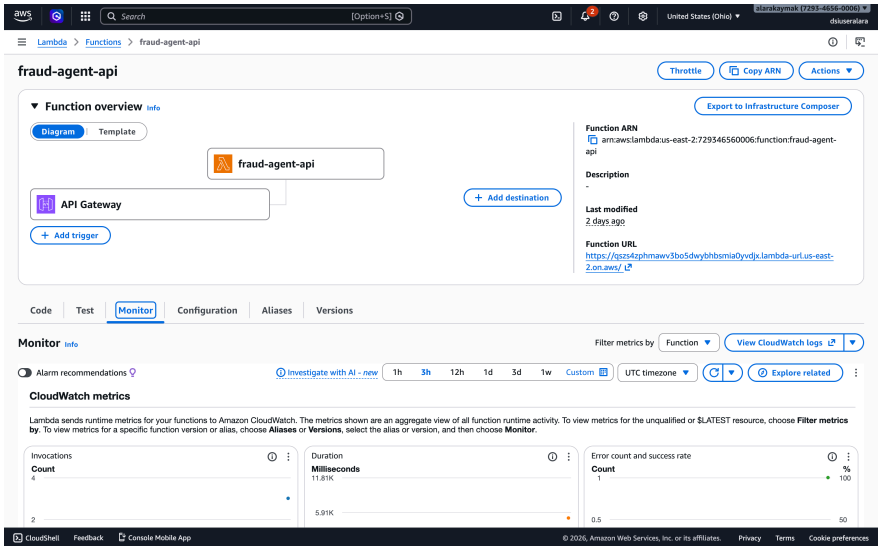

Figure 7: AWS Lambda function overview showing API Gateway trigger and CloudWatch monitoring

i. Reflection

Going into this project I assumed the hardest part would be getting the LLM to make good decisions. That turned out not to be the case – Claude handled the specialist reports well once they were structured correctly. The harder problems were all on the infrastructure side: getting Lambda to accept the container image, figuring out why the model would not load (xgboost was missing from the Lambda environment), and dealing with CORS when the frontend and API were on different origins. A lot of time went into things that had nothing to do with the AI itself.

The evaluation results were also surprising. I expected the LLM to give different answers every time, but it was only inconsistent on the cases where even I was not sure what the right call should be. For anything obvious – a 0.92 score at 3 AM, or a user who always makes large purchases in the same city – it gave the same answer every run.

The biggest gap identified during evaluation was the transaction history data. The velocity agent failed every burst-pattern test case because the test users had no recent burst history in DynamoDB at the time of evaluation — the seeded timestamps had expired out of the 10-minute velocity window. This was subsequently fixed by re-seeding the data with fresh timestamps, and velocity detection now works correctly. The agent logic was always right; it just had nothing to work with at evaluation time.

I would also rethink running all four agents for every transaction. A score of 0.38 goes through the same pipeline as a score of 0.95, which means paying full latency on transactions that are probably fine. A staged approach where you only escalate to more agents if the initial signal is unclear would make the system faster without really changing the quality of decisions where it matters.